Run Pipeline Units With Status Capture

Description

Runs one isolated pipeline store per unit, continuing after unit failures while retaining an explicit status row for every requested unit.

Usage

```
run_pipeline_units_with_status(
  scale_ids = NULL,
  sel_script = NULL,
  run_pipeline_function = run_pipeline,
  progress = TRUE,
  ...,
  verbose = TRUE,
  expected_infeasible_scale_ids = base::character(),
  unit_target_names = NULL
)
```

Arguments

`scale_ids`	Non-empty character vector of unique spatial unit identifiers.
`sel_script`	Single pipeline script path passed to 'run_pipeline_function'.
`run_pipeline_function`	Injectable pipeline runner called with 'sel_script', 'store_suffix', and arguments supplied through '...'. Defaults to [run_pipeline()].
`progress`	Logical scalar controlling the progress indicator. Defaults to 'TRUE'.
`...`	Additional named arguments passed to 'run_pipeline_function'.
`verbose`	Logical scalar controlling per-unit status messages. Defaults to 'TRUE'.

`expected_infeasible_scale_ids`	Units with current preparation evidence proving no model is possible. These units are recorded but never sent to the pipeline runner.
`unit_target_names`	Optional named list of resolution-aware final target requests by unit. Empty requests are recorded as expected infeasible, not executed.

Details

This helper is intended for post-selection full-unit execution. Tuning uses its own status capture and excludes failed stores from tier aggregation. Registered infeasibility discovered during execution is retained as 'expected_infeasible'; unclassified failures remain 'error'.

Value

Tibble with one row per requested unit and columns 'scale_id', 'pipeline_status' ('"ok"', '"expected_infeasible"', or '"error"'), and 'error_message'.

Examples

```
## Not run:

run_pipeline_units_with_status(

  scale_ids = c("eu_r01", "eu_r02"),

  sel_script = "R/Pipelines/pipeline_paleo_spatial_resolution.R",

  prebuild_interpolation = TRUE

)



## End(Not run)
```